

APEX AI Automated Penetration Test Report

Generated on 2026-10-08 16:27:41 | Target: http://127.0.0.1:5000/

Target Host:	http://127.0.0.1:5000/	Overall Risk Rating:	HIGH
Confirmed Vulnerabilities:	2 security finding(s)	Discovered Endpoints:	2 pages & endpoints

Target Application Profile & Context

Application Type:	Search & User Portal
Identified Purpose:	Interactive demonstration application with search and profile features
DOM Architecture:	Search input field, 2 interactive forms, Script tags with innerHTML

Vulnerability Summary Table

Vulnerability Type	Endpoint	Parameter	Severity	Confidence
Reflected Cross-Site Scripting (Reflected XSS)	http://127.0.0.1:5000/search	q	HIGH	90%
DOM-Based Cross-Site Scripting (DOM XSS)	http://127.0.0.1:5000/?callback=hello	callback	HIGH	85%

In-Depth Vulnerability Analysis & Technical Remediation

Finding #1: Reflected Cross-Site Scripting (Reflected XSS)

Finding:	#1: Reflected Cross-Site Scripting (Reflected XSS) [HIGH]
Target URL:	http://127.0.0.1:5000/search
Parameter:	q
Trigger Payload:	alert(1)
Plain Description:	The web application takes user input from an HTTP request (such as a URL query parameter or form submission) and immediately reflects it back in the HTTP response HTML without HTML encoding.
Root Cause:	Missing contextual output encoding: The server reflects input received from the user directly into the HTML response body without converting dangerous HTML characters (<, >, ", ', &) into their safe HTML entity representations.
Trigger Mechanism:	1. The attacker creates a malicious URL containing an HTML/JS payload: 'alert(1)'. 2. A user clicks the link, sending the HTTP request containing parameter 'q' to the web server. 3. The server processes the request and mirrors the unescaped parameter value directly into the HTML template. 4. The victim's web browser renders the response, interprets the payload as code, and executes the injected JavaScript.
Security Impact:	Hijacking victim accounts by stealing cookies and session identifiers, phishing for credentials by injecting fake login prompts, and redirecting users to malicious external domains.
Remediation:	Apply strict contextual output encoding (e.g. htmlspecialchars() in PHP, Jinja2 auto-escaping in Python, or OWASP Java Encoder). Deploy a restrictive Content Security Policy (CSP) with 'unsafe-inline' disabled.

Finding #2: DOM-Based Cross-Site Scripting (DOM XSS)

Finding:	#2: DOM-Based Cross-Site Scripting (DOM XSS) [HIGH]
Target URL:	http://127.0.0.1:5000/?callback=hello
Parameter:	callback
Trigger Payload:	alert(1)
Plain Description:	Client-side JavaScript code takes data from a user-controllable source (like the URL query string or location hash) and writes it directly into an unsafe sink in the Document Object Model (DOM) without proper sanitization.
Root Cause:	Unsafe client-side sink usage: Client-side JavaScript reads user-controlled values from the DOM (e.g., location.search) and writes them directly into execution sinks like element.innerHTML, document.write(), eval(), or setTimeout().
Trigger Mechanism:	1. The victim visits a crafted link containing malicious JavaScript syntax in parameter 'callback'. 2. The client browser downloads and parses the page; client-side JavaScript reads 'callback' from the URL. 3. The script assigns the raw value into a dangerous DOM property (e.g. innerHTML or eval()) without HTML escaping. 4. The browser parses the injected string as live HTML/JavaScript, immediately executing the script in the victim's session context.
Security Impact:	The attacker can steal active session cookies and tokens (session hijacking), log victim keystrokes, perform unauthorized actions on behalf of the logged-in user, and deface the page in the victim's browser.
Remediation:	Avoid writing untrusted data directly to innerHTML. Use safe DOM APIs like element.textContent or element.innerText instead. If HTML rendering is necessary, sanitize the input using a battle-tested library like DOMPurify.

Discovered Application Endpoints

http://127.0.0.1:5000/search
http://127.0.0.1:5000/users?id=1